Synthetic Library Database Design and Implementation

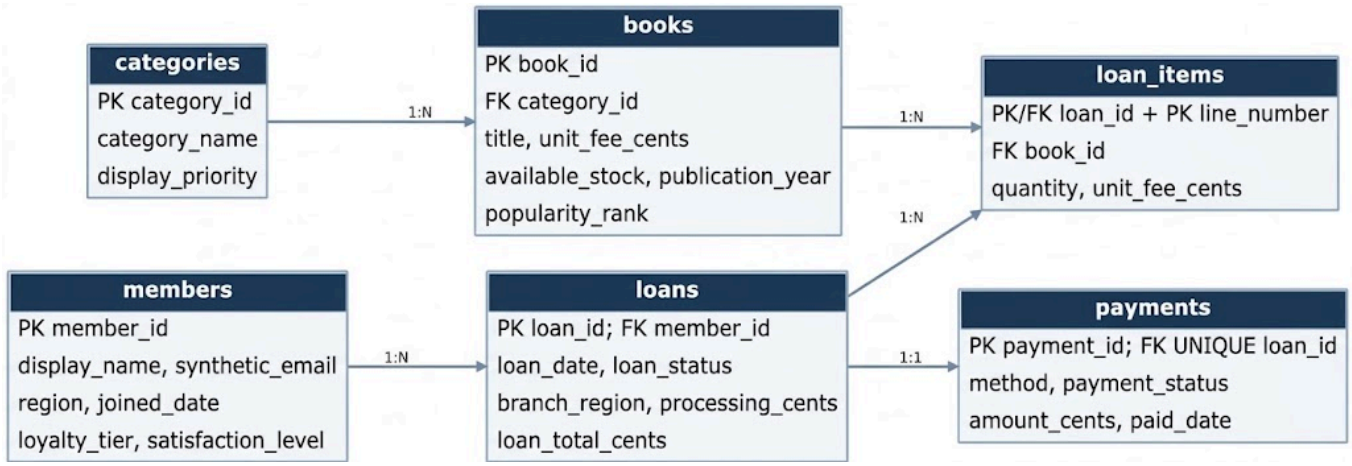
Name Iresha

1. Introduction

This report explains the design, implementation, and testing of a reproducible, relational SQLite database for a synthetic library management system. An aim was to build a structured repository to store the basic entities of a library, such as members, catalogue books, loans, individual loan items, payment transactions etc., to get the structured information and analyze it. All records were created on the fly, not depending on any external data sets.

2. Database Design and Schema

The design of the database was optimized for quick data access, low data duplication, and consistency between related tables(Javed & Lin, 2020). Different business units member data, book information, loan data and transaction data—are processed separately. A primary key is used to identify each record while a foreign key is used to ensure referential integrity and to create relationships. Of particular interest is that the 'loan_items' table has a composite primary key to deal with more than one book for each loan record. This relational structure ensures that data is not duplicated, and allows for complex SQL queries to extract and analyze data.



2.1 Table Composition

Table	Rows	Purpose / Principal Relationship
categories	10	Catalogue classification; referenced by books
members	500	Synthetic member profiles; referenced by loans
books	150	Library inventory with stock and publication data
loans	1,500	Borrowing records linked to members

loan_items	~3,696	Individual books in a loan; links loans and books
payments	1,500	Payment record per loan; loan_id is UNIQUE

2.2 Measurement Scales and SQL Constraints

Scale	Implemented Example	Interpretation
Nominal	members.region (TEXT)	Unordered regions: North, South, East, West, Central
Ordinal	members.satisfaction_level (TEXT)	Ordered: Low, Medium, High, Very High
Interval	books.publication_year (REAL)	Calendar years; zero does not indicate an absence of time
Ratio	loan_items.quantity (INTEGER)	Count with a true zero; positive quantities required

It has INTEGER, REAL, and TEXT storage types, with the addition of NOT NULL, UNIQUE, CHECK, and REFERENCES constraints to ensure data integrity. These constraints check if the money value is positive, the stock value is non-negative, the category value is valid, and the borrowing limit (1-5 books) is valid. To prevent floating-point precision issues when performing financial sums, monetary values are stored as integer cents.

3. Synthetic Data Generation and Implementation

The instantiation of the database was done via python's standard libraries: sqlite3, random and datetime. Records were generated using a fixed random seed (20261106) so that records can be reproduced. The schema was populated with synthetic data using parameterized INSERT operations.Synthetic data was used for the schema using parameterized INSERT operations. Member loyalty tiers, satisfaction levels and loan statuses were weighted randomly assigned. Inventory, processing fees, quantities and dates were produced in a logical range. Importantly, temporal logic was imposed, such that member join dates are before their corresponding loan dates. Loan totals are dynamically calculated from the sum of the item subtotals and the processing fees for active/completed loans, and cancelled loans have a zero charge(Liu et al., 2024).

4. Testing and Validation

The database was then validated rigorously afterward to ensure structural integrity, constraint enforcement and proper business logic(Li et al., 2023).

Validation Check	Result / Output
Required Data Volume	`Loans: 1500 rows; loan_items: 3726 rows`
Database Integrity	`PRAGMA integrity_check returned 'ok'`
Referential Integrity	`PRAGMA foreign key check found 0 violations`

Business Logic	`--- Business Logic ---`
	`Status: Active | Payment: Pending | Count: 306 | Total Due: 756734¢ | Paid: 756734¢`
	`Status: Cancelled | Payment: Void | Count: 155 | Total Due: 0¢ | Paid: 0¢`
	`Status: Completed | Payment: Paid | Count: 1039 | Total Due: 2460246¢ | Paid: 2460246¢`

Constraint Enforcement	`Invalid insert rejected: CHECK constraint failed: quantity BETWEEN 1 AND 5`

Aggregated queries were also run to provide business intelligence information, for example, transaction pattern per region and most borrowed materials(Zeighami et al., 2024).

```
Transaction Patterns by Region (Completed Loans):
('East', 242, 5638.22)
('North', 217, 5144.68)
('West', 200, 4814.5)
('Central', 194, 4644.88)
('South', 186, 4360.18)

Top 5 Most Borrowed Books (Quantities):
('Book Title 49', 119)
('Book Title 133', 119)
('Book Title 89', 115)
('Book Title 53', 115)
('Book Title 52', 115)
```

5. Ethical and Data Privacy Considerations

Everything in this database is completely fictional. No accidental communication with real individuals is made possible by the use of the reserved 'example.com' domain for emails generated(Alimohammadi et al., 2024). The names, areas and behaviour are constructed and not indicative of the real-world situation. For academic demonstration, this would be acceptable but a production environment may require rules for access, encryption of sensitive PII, and 'privacy by design' rules.

6. Conclusion

This project successfully utilized the concepts of relational database to develop a structured library management system with SQLite and Python. The implementation reflects the fact that database normalization, data typing and strict entity relationships are important. The synthetic generation script facilitates the simulation of library operations with a reproducible environment while preserving data privacy of personal information. Moreover, the validation phase showcased the effectiveness of SQL queries to audit data integrity and to obtain aggregated analytical insights.

References

1. Alimohammadi, K., Wang, H., Gulati, O., Srivastava, A. and Azizan, N., 2024. Differentially Private Synthetic Data Generation for Relational Databases. *arXiv preprint arXiv:2405.18670*.
2. Javed, M. and Lin, Y., 2021. iMER: Iterative process of entity relationship and business process model extraction from the requirements. *Information and Software Technology*, *135*, p.106558.
3. Li, J., Zhao, Z., Chundawat, V., Sikdar, B. and Tay, Y.C., 2023. IRG: Generating Synthetic Relational Databases using Deep Learning with Insightful Relational Understanding. *arXiv preprint arXiv:2312.15187*.
4. Liu, S., Tian, C., Sun, J., Wang, R., Lu, W., Zhao, Y., Xue, Y., Wang, J. and Du, X., 2024. Conformance Testing of Relational DBMS Against SQL Specifications. *arXiv preprint arXiv:2406.09469*.
5. Zeighami, S., Seshadri, R. and Shahabi, C., 2023. A neural database for answering aggregate queries on incomplete relational data. *IEEE transactions on knowledge and data engineering*, *36*(7), pp.2790-2802.

Appendix: Code for database creation and tables creation
https://github.com/IreshaShanika/Library_Database_Academic.git